CSC 345 Report for Project 3
Full Names: Paris Watznauer and Anthony Carvajal
Section: 02

Implementations: We implemented every requirement for the project in our programs.

Breakdown of main1.c

```
/* 1. Read in input logical addresses*/
while(fscanf(addresses, "%d", &logical_address) == 1) {
    ++total_addresses;

    /* 2. Translate the input addresses into physical addresses*/
    int page_number = (logical_address >> 8) & 0xFF;
    int offset = logical_address & 0xFF;

    int frame_number = -1;

    /* 6. Count number of TLB hits */
    for (int i = 0; i < 16; i++) {
        if (tlb_page[i] == page_number) {
            frame_number = tlb_frame[i];
            tlb_hits++;
            break;
        }
    }
```

Explanation: After setting up the required files, arrays, and variables, the first 6 project requirements are implemented in main1.c. The program reads an address from addresses.txt one at a time. The page number and offset are calculated using each read address. Next, the TLB is searched to see if it contains the proper translation for the current page number, and it keeps track of how many TLB hits there are.

```
/* 2. Translate the input addresses into physical addresses*/
if (frame_number == -1) {
    frame_number = page_table[page_number];

    /* 5. Counter number of page faults*/
    if (frame_number == -1) {
        page_faults++;
        frame_number = next_frame;
        next_frame++;

        fseek(backing_store, page_number * 256, SEEK_SET);
        fread(physical_memory[frame_number], 1, 256, backing_store);

        page_table[page_number] = frame_number;
    }

    /* 4. FIFO-based TLB update*/
    tlb_page[tlb_idx] = page_number;
    tlb_frame[tlb_idx] = frame_number;
    tlb_idx = (tlb_idx + 1) % 16;
}

/* 2. Translate the input addresses into physical addresses*/
int physical_address = (frame_number << 8) | offset;

/* 3. Retrieved the values stored in the physical addresses*/
signed char value = physical_memory[frame_number][offset];
```

If the address mapping is not found in the TLB, the program then must search the page table. The program searches the page table, keeps track of the number of page faults, and updates the page table based on the current frame number. The page number and frame number are saved in the parallel TLB arrays. The tlb_idx calculation will wrap the index around to 0 once the full 16 indexes have been used, thus implementing FIFO TLB update. Finally, the physical address is computed, and the values stored at the physical address are retrieved.

Output of main1.c:

```
tcnj@tcnj-osc:~/Desktop/project3$ ./main1 addresses.txt
Page Faults: 244
Page Fault Rate: 0.244
TLB Hits: 54
TLB Hit Rate: 0.054
```

Breakdown of main2.c

```
/*tlb miss*/
if (frame_number == -1) {
    frame_number = page_table[page_number];

    if (frame_number == -1) {
        page_faults++;
        frame_number = next_frame;

        int old_page = fr[next_frame];
        if (old_page != -1) {
            page_table[old_page] = -1;

            for (int i = 0; i < 16; ++i) {
                if (tlb_page[i] == old_page) {
                    tlb_page[i] = -1;
                    tlb_frame[i] = -1;
                    break;
                }
            }
        }
```

Explanation: main2.c is very similar to main1.c however physical memory has been limited to only 128 frames, and FIFO page replacement has been implemented in accordance with requirement 7. The program checks if physical memory is full and a page must be evicted. The program invalidates the associated page table index so that subsequent accesses will trigger a page fault, and also invalidates the TLB index associated with the evicted frame if

it exists. Associated information is updated, and the next frame queue pointer is assigned to its next frame position. A similar formula is used as in the FIFO TLB update to ensure the frame allocation order is a circular loop.

Output of main2.c:

```
tcnj@tcnj-osc:~/Desktop/project3$ ./main2 addresses.txt
Page Faults: 538
Page Fault Rate: 0.538
TLB Hits: 54
TLB Hit Rate: 0.054
```

Breakdown of main3.c:

- **Source Code: Timestamp Incrementer:**

```
126
127             /*fifo tlb update*/
128             tlb_page[tlb_idx] = page_number;
129             tlb_frame[tlb_idx] = frame_number;
130             tlb_idx = (tlb_idx + 1) % 16;
131         }
132
133
134         timeForAccess++; //updates the timestamp on every access, regardless if theres a tlb hit or not
135         last_used[frame_number] = timeForAccess;
```

- **Source Code: Victim Search:**

```
92                  int lru_frame = 0;
93
94                  for (int i = 1; i < 128; i++) //finds the frame with the oldest time signature
95                  {
96                      if (last_used[i] < last_used[lru_frame])
97                      {
98                          lru_frame = i;
99                      }
100                 }
101
102                 frame_number = lru_frame;
```

Output of main3.c: Requirement 8

```
tcnj@tcnj-osc:~/Desktop/project3$ ./main3 addresses.txt
Page Faults: 539
Page Fault Rate: 0.539
TLB Hits: 54
TLB Hit Rate: 0.054
```

Explanation: As mentioned above, main3.c is built from the foundation in main1.c. Unlike main1, we opt to limit our physical memory to 128 frames and implement the assigned Least Recently Used page replacement Algorithm. Two arrays are implemented in order to achieve this which are frame_toPage: which records which page is stored in each frame, and last_used records when each frame was last accessed during execution. Then we implemented a counter, timeforAccesss, that increments by one for every address. The code was structured in such a way that once the frame for an address has been found, whether it be a TLB hit, a page table hit, or a page fault, the frame's last_used entry is then set to the current counter value. Unlike FIFO, this process occurs during every access and not just the page faults which makes the algorithm uniquely LRU.

When a page fault does occur our free frames are spent in order while the variable next_frame is below 128. Once the memory becomes full, the program searches last_used for the smallest value, that frame was used the longest time ago so it is to be evicted. The mechanisms that follow are setting the page table entry to -1 so accessing it a later time results in a page fault, and any TLB entry for the page is removed so a stale TLB hit can not return a frame that now holds different data. Then the new page is loaded into the frame, and the page table and frame_toPage are updated.

Breakdown of plotting.py: Requirement 9

- **Source Code: plotting.py:**

```python
#Experiemnt comparing FIFO AND LRU page replacement algo

import subprocess
import matplotlib.pyplot as plt

FRAME_COUNTS = [32, 64, 128, 256, 512, 1024] # different frame count nums that are being tested doubling starting from 32
ADDRESS_FILE = "addresses.txt"

#this is the most important section that prevents the needs of making tedious changes to our mains
#main2.c & main3c have 128 frames written in to the code, this defintion makes a copy of our mains where every
#128 is swapped for the frame count we want to test.
def run_with_frames(source_file, frames):
    with open(source_file) as f:
        code = f.read()
    code = code.replace("128", str(frames))
    with open("temp.c", "w") as f:
        f.write(code)

    subprocess.run(["gcc", "temp.c", "-o", "temp"], check=True) #stopper if theres compiling failure

    output = subprocess.run(["./temp", ADDRESS_FILE], capture_output=True, text=True).stdout

    faults = 0
    rate = 0

    #part of output
    for line in output.split("\n"):
        if line.startswith("Page Faults:"):
            faults = int(line.split(":")[1])

        if line.startswith("Page Fault Rate:"):
            rate = float(line.split(":")[1])

    return faults, rate

#creation of lists to collect the results as the script is executed
fifo_faults = []
fifo_rates = []
lru_faults = []
lru_rates = []

for frames in FRAME_COUNTS:
    faults, rate = run_with_frames("main2.c", frames) #FIFO
    fifo_faults.append(faults)
    fifo_rates.append(rate)

    faults, rate = run_with_frames("main3.c", frames) #LRU
    lru_faults.append(faults)
    lru_rates.append(rate)

for i in range(len(FRAME_COUNTS)):
    print(f"Frames: {FRAME_COUNTS[i]}, FIFO faults: {fifo_faults[i]}, FIFO rate: {fifo_rates[i]:.3f} | LRU faults: {lru_faults[i]}, LRU rate: {lru_rates[i]:.3f}")

labels = [str(frames) for frames in FRAME_COUNTS]
fig, (left, right) = plt.subplots(1, 2, figsize=(12, 5)) #allows for two charts to be created and put side by side

left.plot(labels, fifo_faults, marker="o", label="FIFO")
left.plot(labels, lru_faults, marker="o", label="LRU")

left.set_xlabel("Number of frames")
left.set_ylabel("Page faults")
left.set_title("Page fault vs number of frames")
left.legend()

right.plot(labels, fifo_rates, marker="o", label="FIFO")
right.plot(labels, lru_rates, marker="o", label="LRU")

right.set_xlabel("Number of frames")
right.set_ylabel("Page fault rate")

right.set_title("Page fault rate vs number of frames")
right.legend()

plt.tight_layout() #stops our labels from overlapping since our result graph is more crowded than project 1
plt.savefig("fifo_vs_lru.png")

print("Saved plot to fifo_vs_lru.png")
```

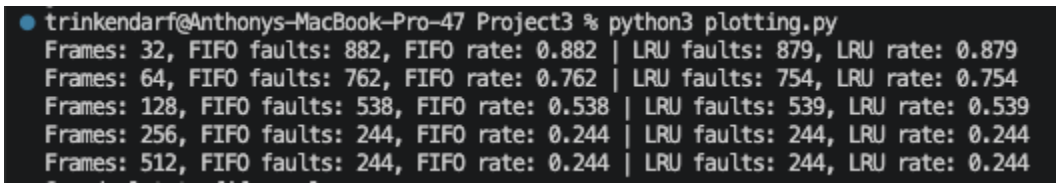
Much like Project 2, we opted to do another Python script that is similar in nature. We were tasked with running the FIFO and LRU algorithm with physical memory sizes of 32, 64, 128, 256, and 512. 5 different sizes in total doubling each time as per the assignment. This script would then record the

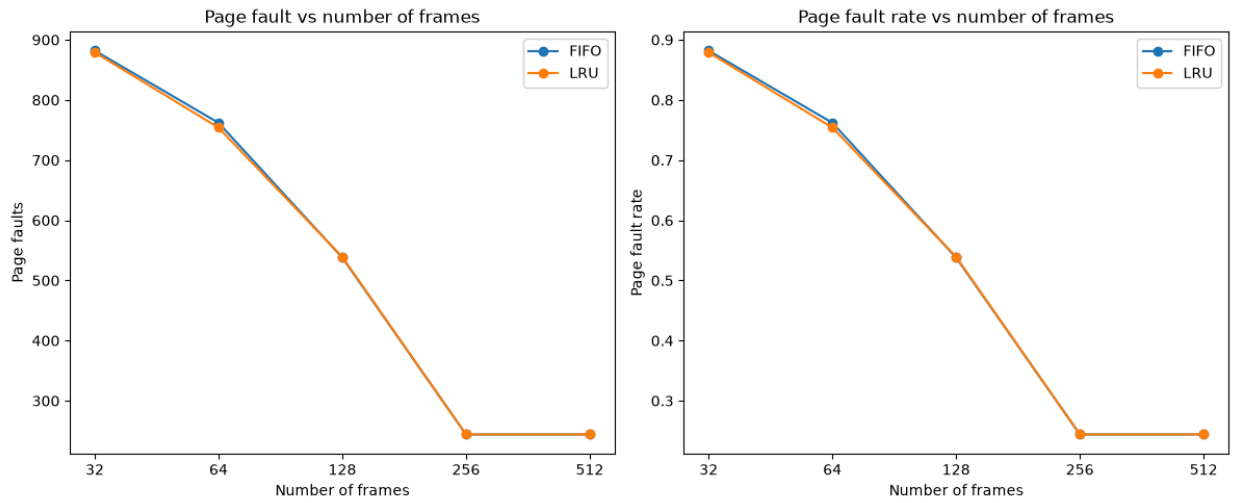
number of page faults and the page fault rate for each of the runs, and finally it would plot the two algos so the results could be compared.

Core logic, both main 2 and 3 have the numbers of frames written into the code as 128. Instead of having to manually change anything within those files, we had a function run-with_frames that would read the source file, replace every 128 with the frame we are testing, and save those results to a temporary copy. This is needed as there is a mismatch between our main c programs and the script. The functionality of mains are locked to a framecount of 128, but we need values from 32-512. So for each frame count, the script makes a copy of main, changes the copy to that frame count, compiles it and then runs.

Furthermore, the main loop goes through each frame count and runs both main2.c and main3.c storing the results in the 4 labeled lists. FIFO faults. FIFO rates, LRU faults, and LRU rates. Each list is written to store the results in the same order as the frame counts, this ensures that the values at the same position all belong to its corresponding test. The final product is the plot reflecting this collected data.

Breakdown and Learning Outcome of Results:





Explanation: We were assigned to compare FIFO and LRU page replacement algorithm performance in main2.c and main3.c respectively. This is done by using the given addresses.txt with physical memory sizes from 32 frames all the way to 512, doubling each time as per the instructions. The left side of the graph shows the number of page faults and the right chart shows the page fault rate.

Both algorithms as can be seen in the visual above show a steady decline in page faults as frames are added. 32 frames: FIFO = 882, LRU = 879, 64 frames: FIFO = 762, LRU = 754, 128 frames: FIFO = 538, LRU = 539. Then something happens at the 256 frames that denotes the moment this behavior changes. At 256 frames both of the algorithms hit a fault of 244, and this does not change as we move towards 512 frames. This behavior is the result of our addresses.txt only using 244 different pages, meaning with 256 frames or up, every page fits in memory and nothing will ever be evicted. The only faults that are remaining are compulsory meaning whichever is the first to access to each page.

As you can see in the visual the algorithms basically overlap at nearly every size. The LRU algorithm only has an advantage over FIFO when recently used pages are likely to be used again soon. We can't take advantage of this since the addresses in our txt file are more random, meaning knowing which pages were recently used doesn't help predict the next access.

Furthermore, LRU and FIFO are roughly equal when it comes to victim selection efficacy so its strong point is moot, this also explains why LRU can come out slightly worse like at 128 frames LRU = 539, FIFO = 538.

Some potential learning outcomes are as follows. For this instance of testing memory size mattered more than the efficiency of the algorithm. When we were doubling the frames it removed hundreds of faults each run, but switching from FIFO to LRU changed the results by at most single digit amounts. Overall, when memory can't hold the pages a program is using, adding memory helps far more than a smarter replacement policy. However, there is a point where more memory does not have an effect. Once every page fits, only compulsory faults occur, meaning adding frames or changings algos make no difference.